

Project 6: Workshop Management & Student Training - AI Service Blueprint

Python + FastAPI + Scikit-Learn + Baseline Fallback Engine

1. AI Service Execution Flow

Client / Node.js Backend -> Request Prediction -> Check Baseline / Fallback Gate -> Execute ML Model / Rule Baseline -> Append Explanation -> Return JSON Response

2. Parts Reorder Prediction API

Item	Value
Method	POST
Endpoint	/ai/predict-reorder
Body	partId, onHandQty, minLevel, maxLevel, weeklyConsumption
Response	200 OK ({ suggestedQty, baselineUsed, explanation })

3. Student Completion Risk API

Item	Value
Method	POST
Endpoint	/ai/student-risk
Body	studentId, attendanceRate, missingAssessments, unmetCompetencies
Response	200 OK ({ riskLevel, completionProbability, explanation })

4. Architecture & Key Components

Component	Purpose
Internal Auth Token	Secures inter-service communication between Node.js and Python.
Moving Average Baseline	Calculates deterministic inventory and risk baselines.
Explainable AI Engine	Appends human-readable justifications to every prediction.
Fallback Safety Gate	Bypasses ML models if data is sparse or service degrades.

5. Fallback & Execution Pipeline

1. Read incoming API JSON request payload.
2. Check data sufficiency and threshold constraints.
3. Trigger deterministic rule baseline if history is insufficient.
4. Run trained Scikit-Learn ML Model if data quality meets release gate.
5. Generate explicit rationale string for output transparency.
6. Return JSON payload with predictions, metadata, and fallback flags.

6. Feature Requirements & Ethical Constraints

- **Inputs:** Inventory logs, consumption rates, student attendance, and task assessments.
- **Strict Exclusion:** Gender, age, nationality, and protected demographic traits are completely excluded from model features to prevent algorithmic bias.
- **Advisory Role:** Predictions remain advisory and can be overridden by human supervisors.

7. Metric Benchmarks & Release Gates

Metric	Benchmark Target
Reorder MAE / WAPE	ML Model must beat Moving Average Baseline on test split
Risk Precision / Recall	>= 80% Precision on 80/20 train/test holdout split
Fallback Uptime Guarantee	100% availability via deterministic fallback rules

8. Status Codes

Code	Meaning
200	OK (Prediction or Baseline Fallback successfully generated)
400	Bad Request (Missing required features in payload)
401	Unauthorized (Invalid internal API key)
422	Unprocessable Entity (Invalid feature data types)
500	Internal Server Error (Service fallback triggered)

9. Error Handling

Scenario	Action
Missing Features	Return 400 Bad Request with missing field details.
Service Timeout	Node.js backend catches timeout and invokes local rule baseline.
Low Data History	Set fallback_active: true and use Moving Average rule.

10. Verification & Test Suite

- **Reorder Test:** Verify stock reorder suggestion for low-inventory parts.
- **Student High-Risk Test:** Confirm risk flag triggers for attendance < 85%.
- **Fallback Override Test:** Simulate AI service crash and verify zero system downtime.

11. Review & Defense Q&A

Q: Why is a deterministic baseline required alongside ML models?

A: To guarantee system operation from Day 1 when history is scarce, satisfying Rule 5.

Q: How does the Explainable AI Engine work?

A: It extracts top decision factors (e.g., "3 missing tasks") into plain text for user trust.

Q: How does fallback guarantee zero downtime?

A: If the Python container fails, Node.js catches the exception and computes rule baselines locally.